

Student aid refund fraud

Detection and prevention platform

Architecture Review Committee | Financial Crimes Enterprise Technology

Response to FinCEN alert

What we are detecting

Several hundred accounts over six months, three distinct fraud typologies

Ghost student

Stolen PII used to enrol fictitious students. The named recipient does not exist or never enrolled.

Identity and enrolment problem

Straw student

A real person sells their identity for compensation. KYC passes cleanly because the person is real.

Behavioural and network problem

Money mule

Refunds are rapidly moved out via cash, P2P, prepaid cards and new business accounts.

Velocity and outflow problem

3

typologies to score

< 1s

scoring latency target

0

code changes to tune logic

Assumptions I designed against

Open questions for stakeholders, and what changes in the design if the answer differs

Enforcement authority

Assumed: we may hold funds and return ACH credits.

If detect only: the core banking path disappears.

Near real time

Assumed: under one second, event driven, not inline authorisation.

If sub-100ms inline: identity calls must be pre-materialised.

Labelled outcomes

Assumed: sparse labels with 90 to 180 day confirmation lag.

If labels are rich: supervised models lead instead of rules.

Enrolment reference data

Assumed: obtainable via Clearinghouse or FSA.

If unavailable: ghost student detection drops to weak proxies.

Entity resolution

Assumed: no enterprise capability exists today.

If one exists: we consume it rather than build it.

Rules platform

Assumed: build on an existing decision engine.

If none: buy versus build becomes a committee decision.

Detection strategy by typology

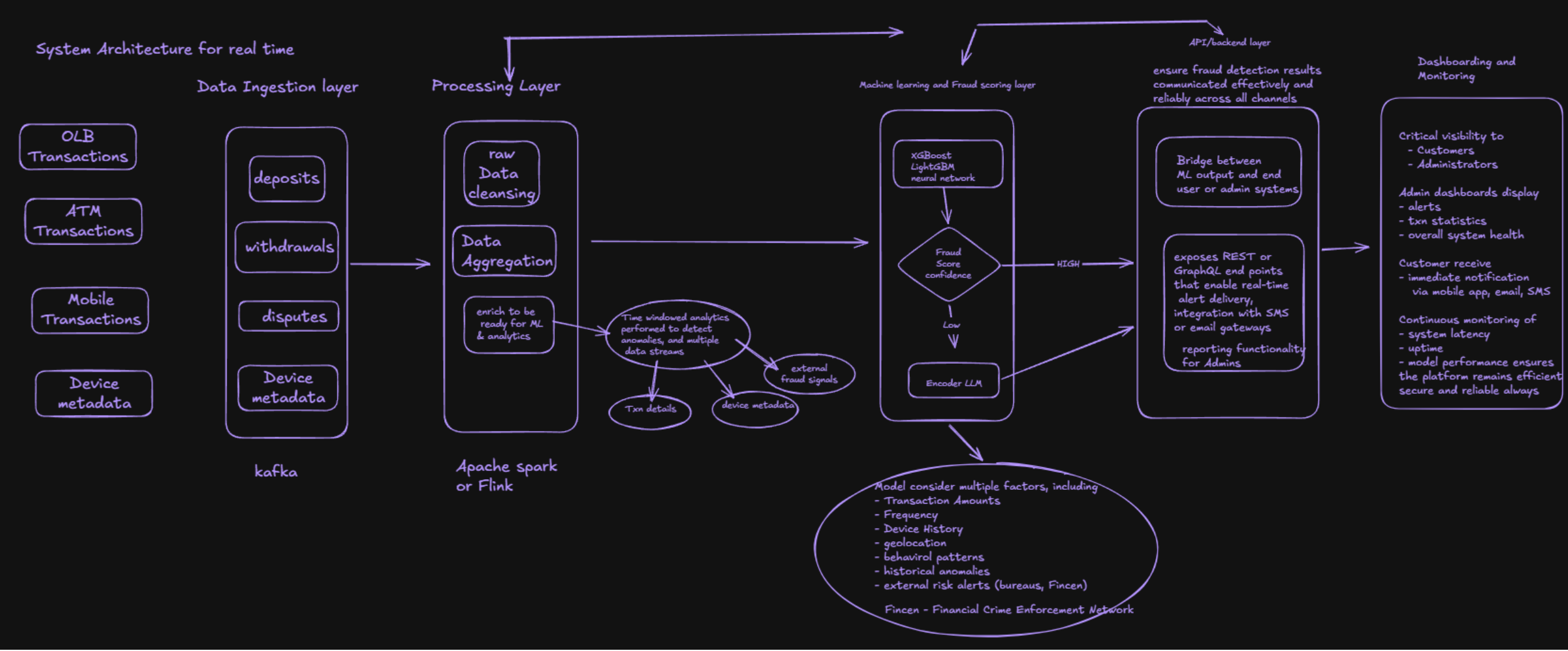
Different problems need different techniques, not one model

Typology	Primary signals	Technique	Key dependency
Ghost student	No enrolment record, thin file, SSN issuance mismatch, minor as recipient	Deterministic rules plus reference data joins	External enrolment data
Straw student	Shared device, address or contact across unrelated recipients, cluster formation	Entity resolution and graph clustering	Entity resolution capability
Money mule	Outflow velocity, cash and P2P share, dormancy then burst, new business payees	Supervised model, gradient boosting	Labelled outcomes and feedback loop

Sequencing: rules and reference joins ship first because the model risk governance clock is shorter, not because they detect better.

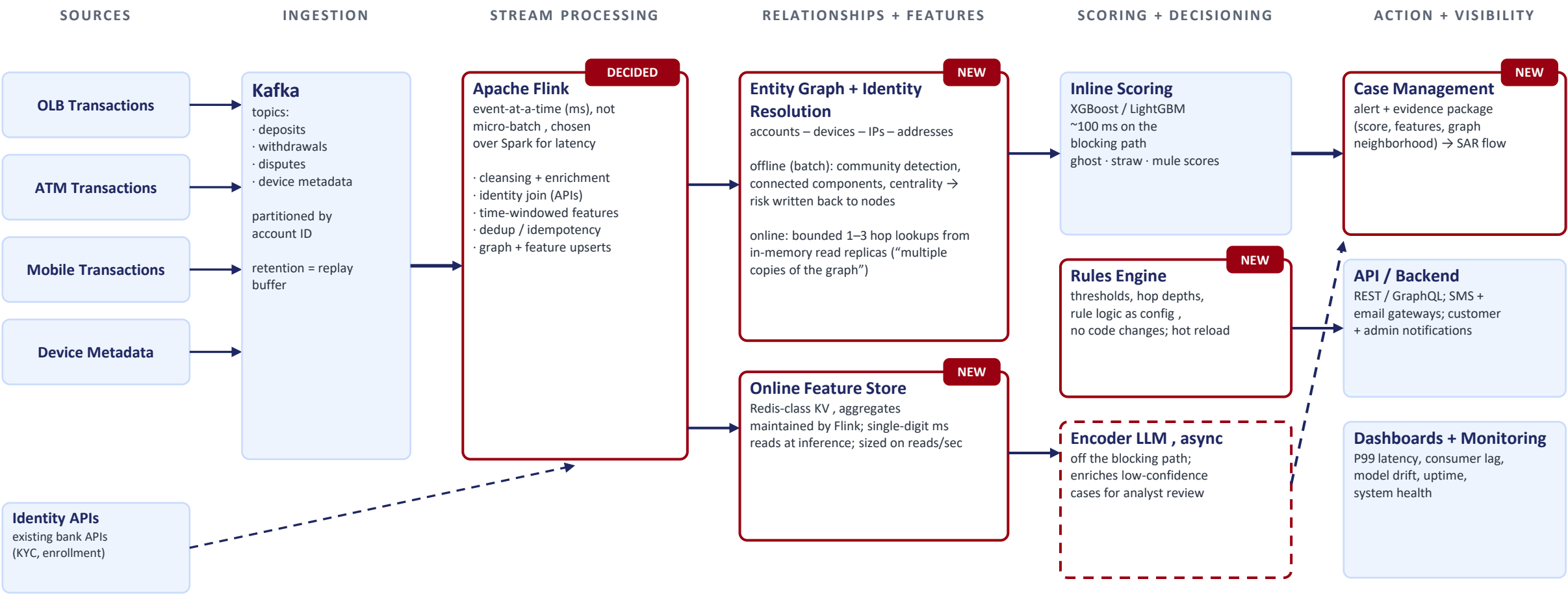
System Architecture Diagram: Fraud Detection

Trying to build a generic reference architecture for the common use cases



Real-Time Fraud Detection , Updated Architecture

Patched: Flink decided over Spark · identity resolution + entity graph · online feature store · LLM moved off the blocking path · case management added



Latency budget (P99 ≤ 500 ms): Kafka ingest ~50 ms → Flink processing ~100 ms → graph + feature lookups ~50 ms → model inference ~100 ms → headroom ~200 ms · LLM + batch graph analytics run off the clock

Scaling: every tier horizontal, Kafka partitions · Flink keyed state (RocksDB + checkpoints) · stateless model servers · graph read replicas · replicated KV feature store

we'd flag when a new account was within two hops of a known fraud cluster

How Graphs Deliver Real-Time Fraud Detection at Scale

FinCEN Student-Aid Fraud Alert · Ghost Students · Straw Students · Money Mules

*Graph theory buys low latency.
Architecture buys scalability.*



Why a Graph Fits Fraud

- Fraud is a network behavior , accounts share devices, IPs, addresses; refunds fan out to mules.
- Index-free adjacency: each node stores its neighbors, so query cost = neighborhood size, not database size.
- Risk scoring becomes a local computation , traverse outward from the event, never rescan the world.



Low Latency: Split Online / Offline

- Offline (batch): community detection, connected components, centrality , results written back as node risk properties.
- Online (per Kafka event): bounded 1–3 hop traversal + precomputed score lookups feed the ML model.
- Heavy graph theory runs ahead of time; the scoring path stays in milliseconds.



Scalability: Architecture

- Read replicas , “multiple copies of the graph”: 35K nodes / 25M edges fits in memory; replicate and load-balance reads.
- Streaming upserts from Kafka mutate the graph incrementally , never rebuild.
- Depth caps + supernode handling bound worst-case query cost, making a latency SLA possible.

Same Graph, Three Detectors



Ghost Students

Connected components over shared PII, devices & IPs collapse hundreds of “different” applicants into one synthetic-identity cluster.



Straw Students

Community detection places real identities inside fraud-ring clusters; recruiter links appear as edges to known ring members.



Money Mules

Directed short-path traversal catches rapid refund → withdrawal hop chains right as the triggering event arrives.



Configurable without code changes: traversal patterns, hop depths, and risk thresholds live as parameterized queries & rules , satisfying the Principal Engineer’s requirement.

System Requirements

Trying to build a generic reference architecture for the common use cases

Functional Requirements

- **Multi-factor authentication and secure login**
The system must support multiple layers of authentication, including biometrics, OTPs, and device verification, ensuring only authorized users can access sensitive banking services.
- **Real-time monitoring of all transactions**
Every transaction , deposits, withdrawals, fund transfers, or payments , must be captured instantly to enable timely analysis and detection of potential fraud.
- **Immediate alerting for high-risk activities**
Suspicious or high-risk transactions must trigger alerts instantly, notifying administrators and customers through dashboards, mobile apps, email, or SMS, enabling prompt action.
- **Admin and customer dashboards with analytics**
Dashboards should provide actionable insights and visual analytics. Administrators can review flagged transactions, while customers can monitor account activity efficiently.
- **Audit trail for regulatory compliance**
The system must maintain comprehensive logs of all interactions, transactions, and alerts, ensuring transparency and enabling regulatory inspections and verification.

System Requirements

Trying to build a generic reference architecture for the common use cases

Non-Functional Requirements

- **Performance: Process transactions in milliseconds**

The system must deliver high-speed processing to detect fraud in real-time, ensuring no delay compromises security.

- **Scalability: Handle spikes in transaction volume**

It should accommodate sudden surges in transaction load, such as festive seasons or promotional campaigns, without affecting system responsiveness.

- **Availability: 24x7 uptime with failover and redundancy**

Continuous system availability is critical, supported by failover mechanisms, redundant servers, and disaster recovery solutions.

- **Security: End-to-end encryption, tokenization, secure APIs**

Comprehensive security must protect sensitive data at rest and in transit, using encryption, tokenization, and secure API connections.

- **Compliance: RBI, SEBI, AML, and KYC regulations**

Regulatory compliance must be embedded in all processes, ensuring adherence to legal requirements and audit standards.

- **Usability: User-friendly dashboards and mobile app interface**

Interfaces should be intuitive, enabling both administrators and customers to access insights, monitor activities, and act on alerts efficiently.

Risk Management Plan

Trying to build a generic reference architecture for the common use cases

- **Cybersecurity breach**

The system faces high risk from potential cyberattacks. To mitigate this, end-to-end encryption, secure firewalls, regular penetration testing, and robust access controls are implemented to protect sensitive financial data from unauthorized access.

- **Model drift / accuracy decay**

Machine learning models can lose accuracy over time due to changes in user behavior or new fraud patterns. Continuous retraining pipelines, monitoring performance metrics, and validating outputs against historical data ensure the models remain effective and reliable.

- **System downtime / outage**

Any interruption in the system could result in delayed fraud detection and financial loss. Redundant servers, load balancing, and automatic failover mechanisms are incorporated to maintain 24x7 availability and minimize disruption.

- **Regulatory non-compliance**

Failure to comply with RBI, SEBI, AML, and KYC regulations can result in penalties and reputational damage. Audit trails, logging, and automated compliance checks ensure all processes adhere to legal requirements.

- **Insider threats / fraud**

Malicious activity by internal staff can compromise security. Role-based access controls, anomaly detection, logging, and regular audits help detect and prevent insider threats effectively.

- **User alert fatigue**

Overloading customers with notifications may reduce responsiveness. Alerts are prioritized, made clear and actionable, and presented through user-friendly dashboards to maintain engagement without causing fatigue.

Business Impact & Strategic Benefits

Trying to build a generic reference architecture for the common use cases

- **Reduced Fraud Losses**

By detecting suspicious transactions in real-time, the system prevents fraudulent activities before they escalate, significantly reducing financial losses for both the bank and its customers.

- **Enhanced Customer Trust and Retention**

Real-time alerts and visible security measures enhance customer confidence. Knowing their accounts are monitored proactively increases loyalty and engagement with banking services.

- **Operational Efficiency**

Automation of fraud detection processes reduces manual investigation workload, allowing staff to focus on high-priority cases while improving overall system responsiveness.

- **Regulatory Compliance and Audit Readiness**

The system maintains comprehensive audit trails, log management, and explainable AI outputs. This ensures that all operations are compliant with banking regulations and ready for inspection at any time.

- **Scalability and Future Readiness**

Designed to handle millions of transactions per day, the system can accommodate growth in user base, transaction volume, and evolving fraud techniques, making it adaptable to future banking needs.

- **Improved Decision-Making and Insights**

Dashboards and analytics provide actionable insights for administrators, enabling data-driven decisions regarding risk mitigation, policy updates, and operational improvements.

Example Case: Pilot implementation in a mid-size bank reduced fraud alerts by **22% false positives** while catching **15% more genuine fraud cases** compared to legacy systems.

Design Lessons & Best Practices: Fraud Detection System



Modular Architecture

Independently scalable components (ingestion, processing ML) allow easy upgrades without disruption



Explainable AI

SHAP & LIME tools provide transparency, reduce bias, and build in automated decisions



Data Versioning & Lineage

Tracking datasets and feature versions ensures auditability, reproducibility, and compliance



Latency vs. Cost Balance

Real-time scoring for high-risk; micro-batches for low-risk. Optimizes performance and infrastructure costs



Compliance by Design

Audit logs, traceability, and regulatory controls embedded to adhere to RBI, SEBI, AML, KYC standards

Conclusion

Trying to build a generic reference architecture for the common use cases

A modern real-time fraud detection system goes beyond technology , it is the backbone of secure, customer-centric banking. By integrating big data pipelines, AI/ML models, and intuitive dashboards, banks can detect and respond to fraudulent activities instantly, minimizing financial losses and operational disruptions.

Such systems also ensure regulatory compliance with standards like RBI, SEBI, AML, and KYC, thanks to embedded audit trails and traceable data flows. Beyond compliance and security, real-time fraud detection enhances customer trust, showing users that their assets are monitored proactively and securely.

Moreover, the modular, scalable, and explainable design allows banks to adapt to evolving threats, upgrade components without downtime, and maintain cost-efficient operations. In an increasingly digital financial landscape, implementing a robust fraud detection platform is not just a technical necessity , it is a strategic advantage that strengthens reputation, operational efficiency, and long-term customer loyalty.

Questions to ponder

- What does "near real time" mean, 100 ms, 500 ms, or a few seconds?
- Is the risk score advisory, or can it place a hold on funds?
- Can we return an ACH credit, or only generate an alert?
- What enrollment/reference data is available and what is its SLA?
- How much historical labeled fraud data exists?
- Does the bank already have an enterprise entity-resolution/graph platform?
- What transaction volume and peak TPS are we designing for?
- What is the acceptable false-positive rate?
- Who owns rule approval and production promotion?
- What regulatory/model-risk constraints apply to the ML model?

Additional material

Why This Problem Is Challenging

Fraud detection is inherently difficult because it involves uncertainty and constantly evolving patterns. Unlike deterministic systems, where outcomes are predictable, fraud detection relies on probabilities and risk assessment.

Challenge	Description	Impact On Design
Real-Time Decisions	Must evaluate transactions instantly	Requires low-latency systems
Evolving Fraud Patterns	Fraudsters adapt quickly	Requires continuous updates
Data Volume	High transaction throughput	Demands scalable architecture

From Simple Validation To Intelligent Systems

In simpler systems, validation might involve checking whether inputs meet certain criteria. Fraud detection goes far beyond this by analyzing behavior, history, and patterns to make decisions.

When designing a fraud detection systems, move from static rules to dynamic decision-making. This shift is what makes the problem both interesting and valuable.

Understanding The Core Goals Of A Fraud Detection System



Before start designing the architecture, need to clearly understand what the system is trying to achieve. Many jump straight into components without defining goals, which leads to unfocused designs.

Detecting Fraud Without Disrupting Users

The primary goal of a fraud detection system is to identify suspicious activity and prevent fraudulent transactions. However, this must be done without negatively impacting legitimate users.

If system blocks too many genuine transactions, it creates friction and damages user experience. This balance between security and usability is a central challenge in fraud detection. → minimize reputation risk

Minimizing False Positives And False Negatives

A key aspect of fraud detection is managing errors, which are categorized as false positives and false negatives. False positives occur when legitimate transactions are flagged as fraud, while false negatives occur when fraudulent transactions are missed. Designing a system that minimizes both types of errors requires careful consideration of thresholds and decision logic.

Error Type	Meaning	Business Impact
False Positive	Legitimate activity flagged as fraud	Poor user experience
False Negative	Fraud not detected	Financial loss

Supporting Investigation And Feedback

Fraud detection systems are not standalone solutions, as they often involve human analysts who review flagged cases. This means system should support investigation workflows and provide relevant data for decision-making. Feedback from these investigations is also critical because it helps improve the system over time. This creates a feedback loop that enhances both rules and models.

Balancing Accuracy And Performance

Another important goal is balancing accuracy with system performance. High accuracy often requires more complex analysis, which can increase latency and cost. When design the system, need to decide how much complexity is acceptable for real-time decision-making. This trade-off is a key discussion point in interviews.

Key Components Of A Fraud Detection Architecture

Once understand the goals, the next step is breaking the system into components. A well-structured architecture makes it easier to explain design and ensures that all aspects of the system are covered.

Thinking In Layers And Components

Fraud detection systems are typically composed of multiple layers that work together to process data and make decisions. Each layer has a specific role, and understanding these roles helps design a cohesive system.

Core Architectural Components

A typical fraud detection system includes several key components that handle different stages of the workflow.

Component	Role	Example Function
Event Ingestion	Captures incoming data	Transaction or login events
Feature Store	Stores historical data	User behavior history
Rules Engine	Applies deterministic logic	Threshold checks
ML Scoring Service	Evaluates risk	Fraud probability scoring
Decision Engine	Determines action	Approve or block transaction

Handling Peak Traffic And Spikes

Fraud detection systems often experience traffic spikes during events like sales or peak business hours. System must be able to handle these spikes without degrading performance.

Auto-scaling mechanisms and buffering strategies help manage these fluctuations. Designing for peak traffic ensures that the system remains reliable under stress.

Multi-Region Deployment And Availability

For global systems, deploying across multiple regions improves both latency and availability. This ensures that users receive fast responses regardless of their location.

However, multi-region deployments introduce challenges such as data consistency and increased costs. Being able to explain these trade-offs is important in interviews.

Building Feedback Loops

Feedback loops are critical for improving both rules and machine learning models. Data from human reviews and confirmed fraud cases can be used to refine detection logic. This continuous learning process ensures that the system adapts to new fraud patterns. It also improves accuracy over time.

Handling Model Drift And Changing Patterns

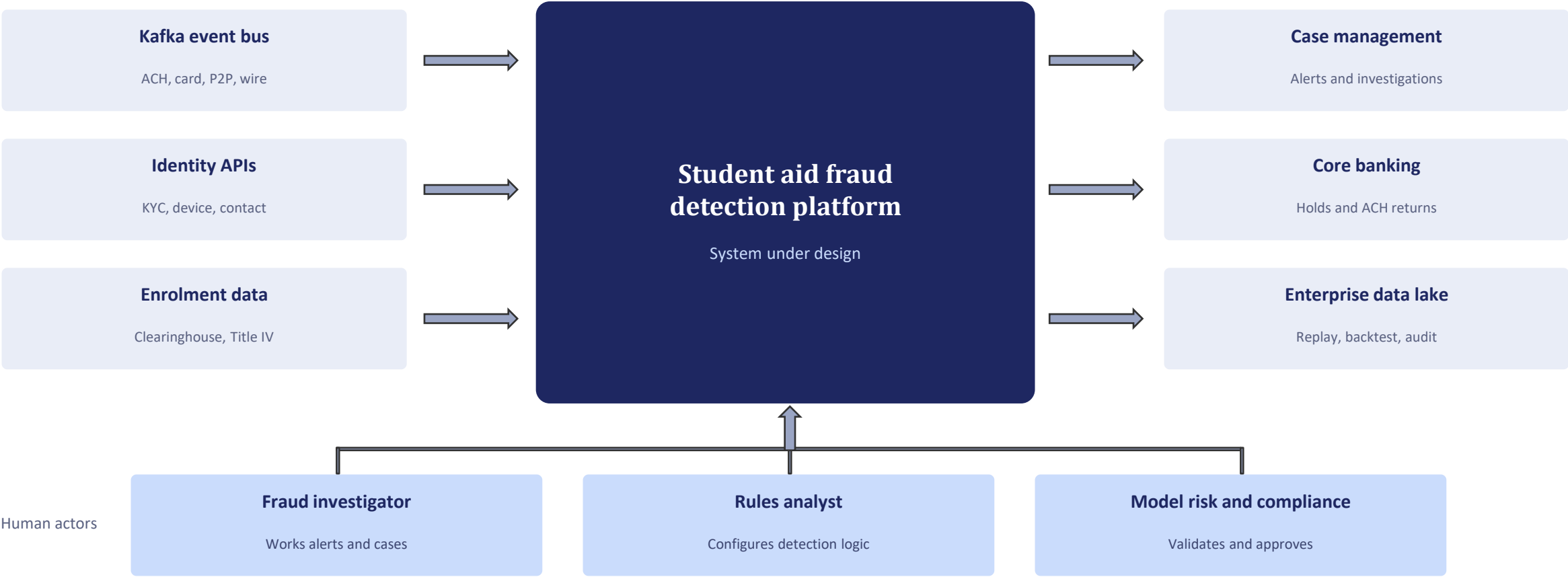
Fraud patterns change as attackers adapt to detection methods. This can lead to model drift, where the model's performance degrades over time. Regular retraining and updating of models help address this issue. This shows that I understand long-term system maintenance.

Interview Insight: Designing For Evolution

Discuss monitoring and feedback demonstrates that beyond initial deployment. It shows that system is designed to evolve and improve over time. This forward-thinking approach is a strong signal of engineering maturity.

Context

System boundary and external dependencies



Target architecture

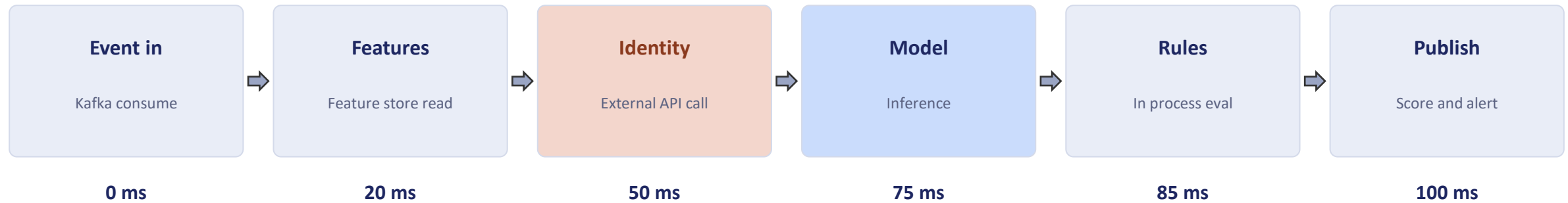
Hot path for scoring, analytical path for training, registry as the only coupling



Training never touches the hot path. The registry is the only coupling, and it is a pull of a versioned artefact.

Near real time path

Illustrative budget , replace with measured figures once the identity API SLA is confirmed



Riskiest dependency

The identity API is the only synchronous external call and is owned by another team. If it degrades, scoring degrades with it.

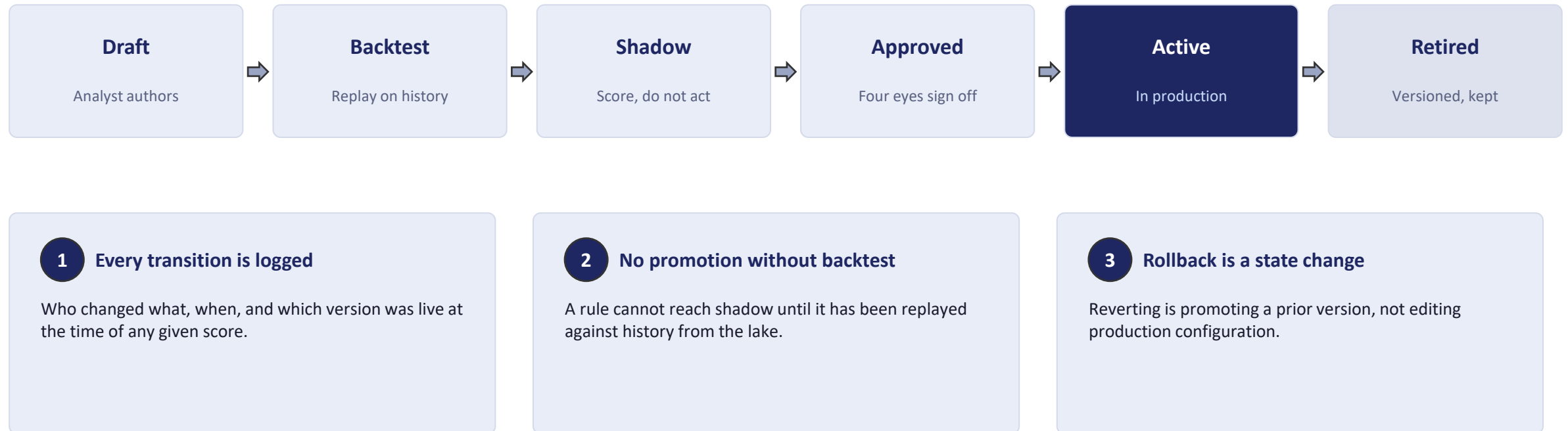
Preferred: hybrid. Cached attributes by default, synchronous call only for new accounts or cold cache , concentrating the expensive call where freshness changes the answer.

Two speed by design

- Per event scoring runs inline in the budget above.
- Network and cluster scoring cannot complete in 100ms , it runs on micro batch and updates an entity risk score.
- The hot path then reads that entity score as a feature.

Rule lifecycle

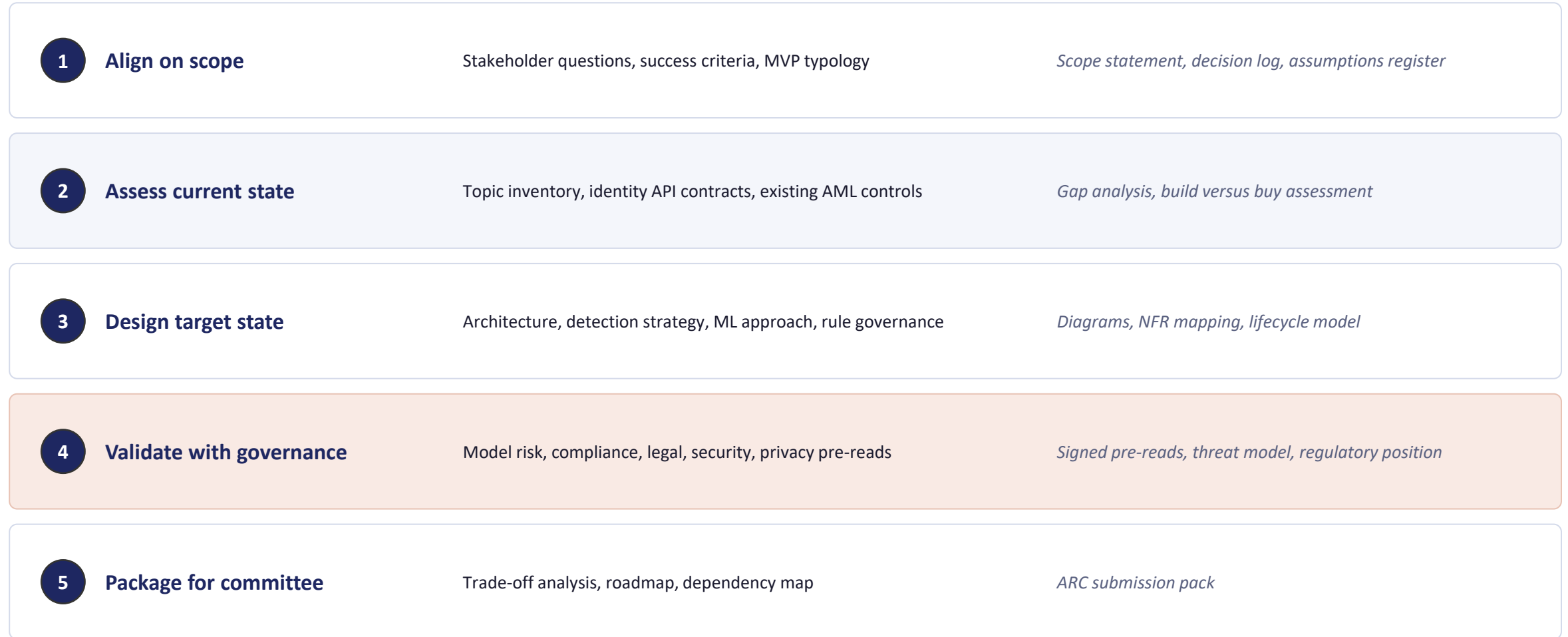
How "configurable without code changes" stays safe and auditable



This is what turns the configurability requirement from an audit finding into an engineering control.

Workflow to review readiness

The process to complete the design, not the runtime data flow



Phases 2 and 3 iterate. What is sequential is the gating: we do not present to committee before governance pre-reads are complete.

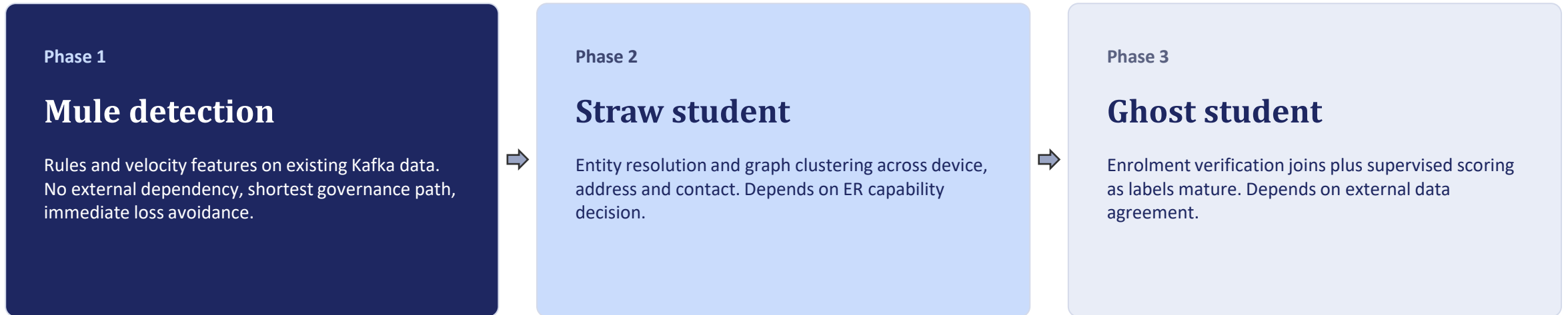
Key decisions and next best alternatives

Every choice here is defensible either way , these are the trade-offs I made

Decision	Next best alternative	Why this way
Rules and reference joins ship before models	Single supervised model across all three typologies	Labels are sparse with long confirmation lag; governance clock on rules is shorter
Model serving as a separate component	Model embedded in the stream processor	Decouples model and application lifecycles; costs one network hop
Model score consumed as a rule input	Rules as a pre-filter, model on survivors	Lets analysts compose model and rule logic without code; costs more inference
Hybrid identity lookup, cached with selective call	Fully synchronous call on every event	Removes an external availability dependency from the hot path for most events

Delivery sequencing

Ordered by governance lead time and data dependency, not by technical appeal



Cross cutting from day one: the analytical path, rule governance, and audit retention of scores and features. These are controls, not phase two features , a regulator asking why an account was not flagged in March needs the score as it existed in March.

Where would I like to go deeper?

Backup

UML sequence diagram with timeout and alt branches

Backup

Kafka topic inventory and data contract questions

Backup

Model risk, fairness testing and reason code approach

Backup

Regulatory position on holds, returns and Reg CC

Open questions and assumptions are documented in the register on slide 3.